Listing 6.4. Evolving strings: putting it all together

```
num_generations = 150
population_size = 900
str_len = len(target)
mutation_rate = 0.00001                                          1

pop_fit = []
pop = spawn_population(size=population_size, length=str_len)     2
for gen in range(num_generations):
    pop, avg_fit = evaluate_population(pop, target)
    pop_fit.append(avg_fit)                                      3
    new_pop = next_generation(pop, \
    size=population_size, length=str_len, mut_rate=mutation_rate)
    pop = new_pop
```

- ***1*** Sets the mutation rate to 0.001%
- ***2*** Creates the initial random population
- ***3*** Records population average fitness over training time

If you run the algorithm, it should take a few minutes on a modern CPU. You can find the highest ranked individual in the population as follows:

```
>>> pop.sort(key=lambda x: x.fitness, reverse=True) #sort in place,
highest fitness first
>>> pop[0].string
"Hello World!"
```

It worked! You can also see the average fitness level of the population increasing each generation in figure 6.6. This is actually a more difficult problem to optimize using an evolutionary algorithm because the space of strings is not continuous; it is hard to take small, incremental steps in the right direction since the smallest step is flipping a character. Hence, if you try making a longer target string, it will take much more time and resources to evolve.

Figure 6.6. This is a plot of average population fitness over the generations. The average population fitness increases fairly monotonically and then plateaus, which looks promising. If the plot was very jagged, the mutation rate might be too high or the population size too low. If the plot converged too quickly, the mutation rate might be too low.